Compte rendu TP N°2

Simulation multi-agent avec Mesa 3

Sujet : formation de l'opinion publique

Ahmed MILOUDI

8 octobre 2026

Objectif du TP

L'objectif est de construire un système multi-agent avec Mesa 3 afin d'étudier la formation d'une opinion publique. Chaque agent possède une opinion comprise entre 0 et 1 et ajuste cette opinion en fonction de ses voisins sur une grille.

Table des matières

1 Description du modèle selon le protocole ODD **3**
1.1 Phénomène étudié . . . 3
1.2 Éléments du modèle . . . 3

2 Implémentation de l'agent **3**

3 Implémentation du modèle **4**
3.1 Correction technique . . . 4

4 Exécution et paramètres **4**

5 Interprétation des résultats **5**
5.1 Rôle du paramètre α . . . 5
5.2 Cas $\alpha = 1.9$ . . . 5
5.3 Consensus et variance . . . 5

6 Réponses aux questions d'analyse **6**
6.1 Pourquoi la polarisation est-elle impossible à long terme ici ? . . . 6
6.2 Comment l'émergence apparaît-elle ? . . . 6
6.3 Activation avec Mesa 3 . . . 6
6.4 Rôle des deux types de reporters . . . 6
6.5 Effet de `torus=True` . . . 6

7 Ouverture : biais de confirmation **6**

8 Ouverture : réseau social au lieu d'une grille **7**

9 Conclusion **8**

Références **8**

1 Description du modèle selon le protocole ODD

1.1 Phénomène étudié

Le modèle représente une population d'individus qui échangent localement leurs opinions. L'objectif est d'observer comment des interactions locales peuvent produire un phénomène global : consensus, dispersion ou groupes d'opinions.

1.2 Éléments du modèle

— **Entités :** les agents représentent les citoyens ;
— **Variable d'état :** l'opinion $o_i \in [0, 1]$;
— **Espace :** une grille torique `MultiGrid` ;
— **Interaction :** chaque agent observe ses voisins de Moore ;
— **Paramètre :** α mesure l'influence sociale ;
— **Observation :** moyenne des opinions, variance et trajectoires individuelles.

La règle d'évolution appliquée à l'agent i est :

$$o_i^{t+1} = o_i^t + \alpha \left(\overline{o}_{\mathcal{N}_i}^t - o_i^t \right) \tag{1}$$

où $\overline{o}_{\mathcal{N}_i}^t$ représente la moyenne des opinions des voisins de l'agent au temps t.

2 Implémentation de l'agent

L'agent initialise son opinion aléatoirement entre 0 et 1. À chaque étape, il récupère ses voisins, calcule leur opinion moyenne, puis se rapproche de cette moyenne selon le coefficient α.

```python
class CitizenAgent(mesa.Agent):
    def __init__(self, model, alpha):
        super().__init__(model)
        self.opinion = self.model.random.random()
        self.alpha = alpha

    def step(self):
        neighbors = self.model.grid.get_neighbors(
            self.pos,
            moore=True,
            include_center=False
        )

        if len(neighbors) > 0:
            mean_opinion = np.mean(
                [neighbor.opinion for neighbor in neighbors]
            )
            self.opinion = (
                self.opinion
                + self.alpha * (mean_opinion - self.opinion)
            )
```

3 Implémentation du modèle

Le modèle crée une grille de largeur `width` et de hauteur `height`. Un agent est placé sur chaque case. Le collecteur `DataCollector` enregistre les indicateurs globaux et individuels.

```
class OpinionModel(mesa.Model):
    def __init__(self, width, height, alpha=0.2, seed=None):
        super().__init__(seed=seed)
        self.grid = mesa.space.MultiGrid(
            width, height, torus=True
        )
        self.alpha = alpha

        for _, (x, y) in self.grid.coord_iter():
            agent = CitizenAgent(self, self.alpha)
            self.grid.place_agent(agent, (x, y))

        self.datacollector = mesa.DataCollector(
            model_reporters={
                "Opinion Moyenne":
                    lambda m: np.mean([a.opinion for a in m.agents]),
                "Variance (Dispersion)":
                    lambda m: np.var([a.opinion for a in m.agents])
            },
            agent_reporters={"Opinion": "opinion"}
        )

    def step(self):
        self.datacollector.collect(self)
        self.agents.shuffle_do("step")
```

3.1 Correction technique

Dans la version de Mesa utilisée, `coord_iter()` renvoie un couple de la forme `(contenu_de_la_case, (x, y))`. La décomposition correcte est donc :

```
for _, (x, y) in self.grid.coord_iter():
    ...
```

4 Exécution et paramètres

La configuration principale utilisée dans le notebook est :

```
WIDTH = 10
HEIGHT = 10
ALPHA = 0.5
STEPS = 50
model = OpinionModel(
    width=WIDTH,
    height=HEIGHT,
    alpha=ALPHA,
```

```
    seed=42
)
```

La grille contient donc $10 \times 10 = 100$ agents. Le paramètre `seed=42` rend l'initialisation aléatoire reproductible.

5 Interprétation des résultats

5.1 Rôle du paramètre α

Le paramètre α contrôle la vitesse à laquelle un agent se rapproche de l'opinion moyenne de ses voisins :

- si α est proche de 0, l'agent change très peu à chaque tick ; la convergence est lente ;
- si $\alpha = 0.5$, l'agent effectue une correction intermédiaire ; le système converge progressivement ;
- si $\alpha = 1$, l'agent adopte directement la moyenne de ses voisins ; la convergence peut être plus rapide.

Pour conserver une opinion dans l'intervalle $[0, 1]$, il est préférable de choisir $\alpha \in [0, 1]$. Dans ce cas, la nouvelle opinion est une combinaison convexe de l'ancienne opinion et de la moyenne des voisins.

5.2 Cas $\alpha = 1.9$

La seconde expérience du notebook utilise $\alpha = 1.9$. Cette valeur est en dehors de l'intervalle annoncé dans le protocole ODD. La mise à jour dépasse alors la moyenne des voisins :

$$o_i^{t+1} = o_i^t + 1.9 \left(\overline{o}_{\mathcal{N}_i}^t - o_i^t \right) \tag{2}$$

L'agent peut donc dépasser la valeur moyenne, osciller et même sortir de l'intervalle $[0, 1]$. Il faut interpréter cette expérience comme un test de sur-correction, et non comme une influence sociale classique. Pour une comparaison cohérente, il serait préférable de tester par exemple $\alpha = 0.1$, 0.5 et 1.0.

5.3 Consensus et variance

La courbe de l'opinion moyenne décrit le comportement global de la population. La courbe de variance mesure la dispersion entre les agents :

- une variance qui diminue indique que les opinions se rapprochent ;
- une variance faible indique un consensus ou un état très homogène ;
- les trajectoires individuelles qui se rapprochent indiquent une convergence collective.

6 Réponses aux questions d'analyse

6.1 Pourquoi la polarisation est-elle impossible à long terme ici ?

Avec $\alpha \in [0, 1]$, chaque mise à jour rapproche l'agent de la moyenne de ses voisins. Il n'existe ni seuil de rejet, ni renforcement de l'écart, ni interaction négative. La règle est donc une règle de moyenne qui réduit progressivement les différences locales. Sur une grille connectée, cette dissipation des écarts conduit généralement à un consensus global.

6.2 Comment l'émergence apparaît-elle ?

Le modèle ne contient pas de coordinateur central qui impose une opinion finale. Chaque agent applique uniquement une règle locale à ses voisins. Pourtant, les graphiques montrent des propriétés globales, comme la baisse de la variance et le rapprochement des trajectoires. Le consensus est donc une propriété émergente : il résulte de nombreuses interactions locales simples.

6.3 Activation avec Mesa 3

L'instruction :

```
self.agents.shuffle_do("step")
```

active les agents dans un ordre aléatoire à chaque tick. Cet ordre évite de favoriser systématiquement les agents placés en premier dans une liste. Il réduit aussi les biais liés à un ordre d'activation fixe et conserve une stochasticité réaliste dans la simulation.

6.4 Rôle des deux types de reporters

— `model_reporters` enregistre une valeur globale, par exemple la moyenne ou la variance de toutes les opinions ;

— `agent_reporters` enregistre l'état de chaque agent, par exemple son opinion individuelle.

Les deux sont nécessaires : les reporters du modèle permettent d'étudier le consensus global, tandis que les reporters des agents permettent d'observer les trajectoires individuelles et d'identifier d'éventuels groupes.

6.5 Effet de `torus=True`

Avec `torus=True`, les bords de la grille sont connectés. Un agent placé en $x = 0$ peut donc avoir comme voisin un agent situé près du bord opposé. La grille se comporte comme une surface qui se referme sur elle-même, ce qui évite un traitement particulier des bords et donne à tous les agents un voisinage comparable.

7 Ouverture : biais de confirmation

Pour introduire un seuil de tolérance ε, l'agent peut ignorer les voisins dont l'opinion est trop éloignée :

```python
class CitizenAgent(mesa.Agent):
    def __init__(self, model, alpha, epsilon=0.1):
        super().__init__(model)
        self.opinion = self.model.random.random()
        self.alpha = alpha
        self.epsilon = epsilon

    def step(self):
        neighbors = self.model.grid.get_neighbors(
            self.pos,
            moore=True,
            include_center=False
        )

        compatible = [
            neighbor for neighbor in neighbors
            if abs(neighbor.opinion - self.opinion)
            <= self.epsilon
        ]

        if compatible:
            mean_opinion = np.mean(
                [neighbor.opinion for neighbor in compatible]
            )
            self.opinion += self.alpha * (
                mean_opinion - self.opinion
            )
```

Cette modification peut produire plusieurs clusters d'opinion. Les agents proches continuent à s'influencer, tandis que les groupes trop éloignés cessent de communiquer. Le système peut alors rester dans une situation de polarisation ou de coexistence de plusieurs groupes.

8 Ouverture : réseau social au lieu d'une grille

Pour représenter un réseau d'abonnés, il serait plus adapté d'utiliser un graphe NetworkX associé à `mesa.space.NetworkGrid`. Les sommets représentent les utilisateurs et les arêtes représentent les relations d'influence. Un influenceur peut être modélisé par un sommet ayant un degré élevé :

```python
import networkx as nx

network = nx.erdos_renyi_graph(
    n=100,
    p=0.05,
    seed=42
)
grid = mesa.space.NetworkGrid(network)
```

Une topologie de type réseau social permettrait d'étudier les hubs, les communautés, les utilisateurs isolés et la propagation d'une information entre groupes.

9 Conclusion

Ce TP montre comment un système multi-agent peut faire émerger un phénomène global à partir de règles locales. Avec une mise à jour par moyenne et un coefficient α compris entre 0 et 1, la dispersion diminue généralement et le consensus apparaît. L'introduction d'un seuil de tolérance, d'influenceurs ou d'une topologie en graphe permettrait d'obtenir des comportements plus riches, comme la polarisation ou la formation de communautés.

Références

— Documentation Mesa : https://mesa.readthedocs.io/
— Dépôt officiel Mesa : https://github.com/mesa/mesa